

## Índice

|                                                 |          |
|-------------------------------------------------|----------|
| <b>Laboratorio 3: Transporte</b>                | <b>1</b> |
| Objetivos                                       | 1        |
| Punto de partida                                | 2        |
| Parámetros base                                 | 2        |
| Parte 1: Análisis con buffers y tasas limitadas | 2        |
| Objetivo                                        | 2        |
| 1. Cambios en <code>network.ned</code>          | 3        |
| 2. Casos de estudio                             | 5        |
| 3. Cambios en <code>Generator.cc</code>         | 5        |
| 4. Cambios en <code>Queue.cc</code>             | 6        |
| 5. Configuración en <code>omnetpp.ini</code>    | 7        |
| 6. Experimentos de la Parte 1                   | 8        |
| 7. Preguntas de la Parte 1                      | 8        |
| 8. Entrega intermedia                           | 9        |
| Parte 2: Control por ventanas TCP-inspired      | 9        |
| Objetivo                                        | 9        |
| 1. Cambios en la red                            | 9        |
| 2. Ventanas obligatorias                        | 10       |
| 3. Paquetes de datos y feedback                 | 11       |
| 4. Pistas prácticas de OMNeT++                  | 12       |
| 5. Señales de congestión                        | 13       |
| 6. Libertades de diseño                         | 14       |
| 7. Experimentos de la Parte 2                   | 14       |
| 8. Preguntas de la Parte 2                      | 14       |
| Competición de control de transporte            | 15       |
| Escenario oficial                               | 15       |
| Métricas oficiales                              | 15       |
| Reproducibilidad                                | 16       |
| Informe                                         | 16       |
| Entrega Final                                   | 16       |
| Criterios de evaluación                         | 16       |
| Consejos y errores frecuentes                   | 17       |
| Comandos útiles                                 | 17       |

## Laboratorio 3: Transporte

Cátedra de Redes y Sistemas Distribuidos - FaMAF

### Objetivos

En este laboratorio van a trabajar con un modelo de red en OMNeT++ para estudiar problemas de transporte. El foco no está solamente en que la simulación corra, sino en usarla para entender qué pasa cuando una red tiene tasas de transmisión acotadas y buffers finitos.

Al finalizar el laboratorio deberían poder:

- Leer, modificar y extender modelos de red en OMNeT++.
- Modelar paquetes con tamaño en bytes y enlaces con tasas de transmisión.
- Medir demora, ocupación de buffers y pérdida de paquetes.

- Distinguir entre problemas de control de flujo y problemas de control de congestión.
- Diseñar una estrategia simple de realimentación entre receptor y transmisor.
- Analizar resultados experimentales y justificar conclusiones en un informe.

### Punto de partida

Se entrega un proyecto kickstarter con tres módulos simples conectados en serie:

Generator -> Queue -> Sink

El comportamiento de cada módulo está implementado en:

- `Generator.cc`: genera mensajes.
- `Queue.cc`: encola mensajes y los atiende con un tiempo de servicio configurable.
- `Sink.cc`: recibe mensajes y registra métricas de demora.
- `network.ned`: define la topología.
- `omnetpp.ini`: define parámetros de simulación.

El modelo inicial es intencionalmente incompleto: no modela tamaño de paquetes, tasas de transmisión ni buffers limitados. La primera parte del laboratorio consiste en agregar esos conceptos para poder analizar el comportamiento de la red.

### Parámetros base

Salvo que se indique otra cosa, usen los siguientes valores:

| Parámetro                                | Valor                |
|------------------------------------------|----------------------|
| Tiempo de simulación                     | 200s                 |
| Tamaño de paquete                        | 12500 Bytes          |
| Intervalo medio inicial de generación    | 100ms                |
| Distribución de generación               | Exponencial          |
| Tamaño de buffers intermedios y receptor | 200 paquetes         |
| Tamaño del buffer transmisor             | Arbitrariamente alto |
| Delay de enlaces entre nodos             | 100us                |

El intervalo de generación debe quedar como parámetro, porque luego se usará para hacer simulaciones paramétricas.

### Parte 1: Análisis con buffers y tasas limitadas

#### Objetivo

Modificar el modelo inicial para estudiar qué ocurre cuando:

- Los paquetes tienen tamaño real.
- Los enlaces tienen `datarate`.
- Las colas tienen capacidad máxima.
- Los paquetes que llegan a una cola llena se descartan.

La red a construir debe tener esta estructura conceptual:

Donde:

- `nodeTx` contiene el generador y un buffer de transmisión.

### Modelo con buffers y tasas limitadas

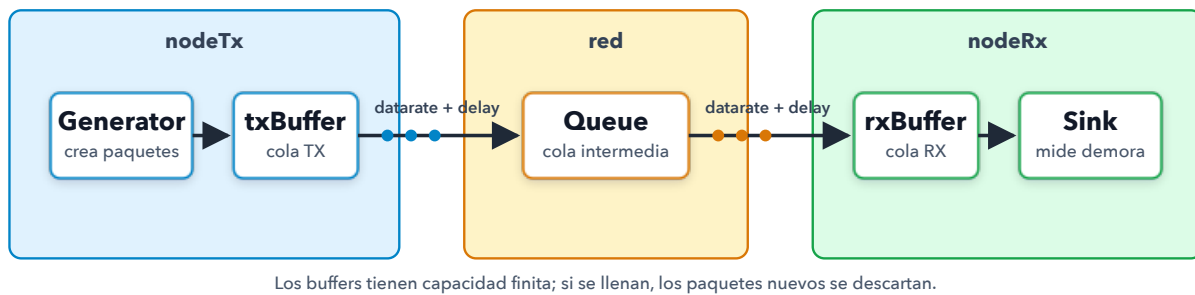


Figure 1: Estructura conceptual de la red de la Parte 1

- queue representa una cola intermedia de la red.
- nodeRx contiene un buffer de recepción y el Sink.

En este laboratorio, la Queue intermedia es una simplificación deliberada: representa una red de múltiples saltos reducida a un único buffer y un único enlace de salida. En una red real, ese tramo podría incluir varios routers, enlaces, colas, políticas de enrutamiento y puntos de congestión. Modelar esa estructura con más detalle corresponde a la capa de red, que será tema de próximos laboratorios. Para este laboratorio nos interesa abstraer todo ese camino como un recurso compartido con capacidad limitada, suficiente para estudiar control de flujo y congestión desde la capa de transporte.

Pueden implementar los buffers reutilizando el módulo Queue del kickstarter, extendido con los cambios pedidos abajo.

#### 1. Cambios en network.ned

Agreguen al generador un parámetro para el tamaño de paquete:

```
simple Generator
{
  parameters:
    double generationInterval @mutable; // segundos
    int packetByteSize;                // bytes
    @display("i=block/source");
  gates:
    output out;
}
```

Agreguen a la cola un parámetro para limitar la cantidad de paquetes:

```
simple Queue
{
  parameters:
    int bufferSize;
    @display("i=block/queue;q=buffer");
  gates:
    input in;
    output out;
```

```
}
```

Una forma simple de definir los nodos compuestos es:

```
module NodeTx
{
    gates:
        output out;
    submodules:
        gen: Generator {
            @display("p=70,50");
        }
        txBuffer: Queue {
            @display("p=70,140");
        }
    connections:
        gen.out --> txBuffer.in;
        txBuffer.out --> out;
}

module NodeRx
{
    parameters:
        double rxToSinkDatarate @unit(bps) = default(1Mbps);
    gates:
        input in;
    submodules:
        rxBuffer: Queue {
            @display("p=70,50");
        }
        sink: Sink {
            @display("p=70,140");
        }
    connections:
        in --> rxBuffer.in;
        rxBuffer.out --> { datarate = parent.rxToSinkDatarate; } --> sink.in;
}
```

En el módulo Network, conviene parametrizar las tasas para poder correr los dos casos desde om-netpp.ini:

```
network Network
{
    parameters:
        double txToQueueDatarate @unit(bps) = default(1Mbps);
        double queueToRxDatarate @unit(bps) = default(1Mbps);
        double linkDelay @unit(s) = default(100us);
    submodules:
        nodeTx: NodeTx;
        queue: Queue;
```

```

    nodeRx: NodeRx;
    connections:
        nodeTx.out --> { datarate = parent.txToQueueDatarate; delay = parent.linkDelay; } --> queue
        queue.out --> { datarate = parent.queueToRxDatarate; delay = parent.linkDelay; } --> nodeRx
}

```

## 2. Casos de estudio

Deben comparar dos escenarios. La diferencia está en donde aparece el cuello de botella.

### Caso 1: posible problema de control de flujo

| Enlace                  | Datarate | Delay          |
|-------------------------|----------|----------------|
| nodeTx -> queue         | 1 Mbps   | 100us          |
| queue -> nodeRx         | 1 Mbps   | 100us          |
| nodeRx.rxBuffer -> sink | 0.5 Mbps | puede omitirse |

En este caso, la red entrega paquetes al receptor más rápido de lo que el receptor los envía a la aplicación (sink). Si hay pérdidas, deberían aparecer principalmente en el buffer de recepción.

### Caso 2: posible problema de control de congestión

| Enlace                  | Datarate | Delay          |
|-------------------------|----------|----------------|
| nodeTx -> queue         | 1 Mbps   | 100us          |
| queue -> nodeRx         | 0.5 Mbps | 100us          |
| nodeRx.rxBuffer -> sink | 1 Mbps   | puede omitirse |

En este caso, el cuello de botella está en un enlace de la red. Si hay pérdidas, deberían aparecer principalmente en la cola intermedia.

## 3. Cambios en Generator.cc

El generador debe crear cPacket, no cMessage, porque los cPacket tienen tamaño en bits/bytes y OMNeT++ puede calcular la duración de transmisión en canales con datarate.

Ejemplo de la parte central de handleMessage:

```

void Generator::handleMessage(cMessage *msg)
{
    cPacket *pkt = new cPacket("packet");
    pkt->setByteLength(par("packetByteSize"));
    pkt->setKind(1); // paquete de datos

    send(pkt, "out");

    simtime_t nextTime = simTime() + par("generationInterval");
    scheduleAt(nextTime, sendMsgEvent);
}

```

También pueden registrar la cantidad de paquetes generados:

```

cOutVector packetGenVector;

void Generator::initialize()
{
    packetGenVector.setName("packetGen");
}

// Al generar un paquete:
packetGenVector.record(1);

```

#### 4. Cambios en Queue.cc

La cola debe:

- Encolar paquetes si hay espacio.
- Descartar y borrar paquetes si el buffer está lleno.
- Registrar la ocupación del buffer.
- Registrar pérdidas por overflow.
- Usar como tiempo de servicio la duración de transmisión del paquete por el enlace de salida.

Agreguen estado para el tamaño máximo y las métricas:

```

class Queue : public cSimpleModule {
private:
    cQueue buffer;
    cMessage *endServiceEvent;
    int bufferSize;
    long droppedPackets;
    cOutVector bufferSizeVector;
    cOutVector packetDropVector;
    // ...
};

```

Inicialicen esos valores en initialize:

```

void Queue::initialize()
{
    buffer.setName("buffer");
    endServiceEvent = new cMessage("endService");

    bufferSize = par("bufferSize");
    droppedPackets = 0;

    bufferSizeVector.setName("bufferSize");
    packetDropVector.setName("packetDrops");
}

```

Al recibir un paquete nuevo, controlen el límite antes de encolar:

```

if (buffer.getLength() >= bufferSize) {
    droppedPackets++;
    packetDropVector.record(droppedPackets);
}

```

```

    delete msg;
    return;
}

buffer.insert(msg);
bufferSizeVector.record(buffer.getLength());

if (!endServiceEvent->isScheduled()) {
    scheduleAt(simTime(), endServiceEvent);
}

```

Cuando termina el servicio, saquen el siguiente paquete, envíenlo y mantengan ocupada la cola durante la duración de transmisión del canal de salida:

```

if (msg == endServiceEvent) {
    if (!buffer.isEmpty()) {
        cPacket *pkt = check_and_cast<cPacket *>(buffer.pop());

        cChannel *channel = gate("out")->findTransmissionChannel();
        simtime_t serviceTime = channel ? channel->calculateDuration(pkt) : SIMTIME_ZERO;

        send(pkt, "out");
        bufferSizeVector.record(buffer.getLength());

        scheduleAt(simTime() + serviceTime, endServiceEvent);
    }
    return;
}

```

En finish, registren al menos la cantidad total de descartes:

```

void Queue::finish()
{
    recordScalar("Dropped packets", droppedPackets);
}

```

## 5. Configuración en omnetpp.ini

Usen configuraciones separadas para que los resultados sean reproducibles. Este ejemplo asume los nombres de módulos sugeridos arriba:

```

[General]
network = Network
sim-time-limit = 200s

Network.nodeTx.gen.packetByteSize = 12500
*.nodeTx.gen.generationInterval = exponential({gi=0.1,0.2,0.5,1.0})

Network.nodeTx.txBuffer.bufferSize = 2000000
Network.queue.bufferSize = 200
Network.nodeRx.rxBuffer.bufferSize = 200

```

```
[Config Caso1]  
description = "Cuello de botella en el receptor"  
Network.txToQueueDatarate = 1Mbps  
Network.queueToRxDatarate = 1Mbps  
Network.nodeRx.rxToSinkDatarate = 0.5Mbps
```

```
[Config Caso2]  
description = "Cuello de botella en la red"  
Network.txToQueueDatarate = 1Mbps  
Network.queueToRxDatarate = 0.5Mbps  
Network.nodeRx.rxToSinkDatarate = 1Mbps
```

Pueden usar otros valores intermedios de `generationInterval`, pero deben cubrir el rango entre 0.1 y 1.0 segundos con suficientes puntos para justificar las conclusiones.

## 6. Experimentos de la Parte 1

Para cada caso de estudio, corran simulaciones variando `generationInterval`.

Como mínimo, generen gráficas representativas de:

- Ocupación del buffer de transmisión (`nodeTx`).
- Ocupación de la cola intermedia (`queue`).
- Ocupación del buffer de recepción (`nodeRx`).
- Cantidad de paquetes descartados, cuando existan pérdidas.
- Carga ofrecida vs. carga útil recibida, en paquetes por segundo.

La carga ofrecida puede estimarse como:

```
paquetes generados / tiempo de simulación
```

La carga útil recibida puede estimarse como:

```
paquetes recibidos por Sink / tiempo de simulación
```

También pueden analizar demora promedio o demora en función del tiempo. En los dos casos de estudio, el comportamiento de la demora puede ser parecido: lo importante de esta parte es identificar en qué buffer aparece el problema.

## 7. Preguntas de la Parte 1

Respondan y justifiquen:

1. ¿Qué diferencias observan entre el Caso 1 y el Caso 2?
2. ¿Cuál es la fuente limitante en cada caso?
3. ¿En qué caso se observa un problema de control de flujo y en cuál uno de control de congestión?
4. ¿Qué evidencia experimental usan para sostener esa conclusión?

Para la discusión conceptual, revisen la diferencia entre control de flujo y control de congestión. Una forma breve de pensarlo:

- Control de flujo: evita que el transmisor sature al receptor.
- Control de congestión: evita que el tráfico agregado sature recursos de la red.



## 8. Entrega intermedia

La Parte 1 concluye con una entrega intermedia. En esta entrega deben presentar el modelo con buffers limitados, paquetes con tamaño, enlaces con tasas configuradas, métricas de ocupación y descarte, y el análisis experimental de los dos casos de estudio.

Esta entrega debe permitir reproducir las simulaciones de la Parte 1 y debe incluir las respuestas a las preguntas de análisis. La Parte 2 se construye sobre esta base.

## Parte 2: Control por ventanas TCP-inspired

### Objetivo

Diseñar una estrategia de control de flujo y congestión inspirada en TCP, pero simplificada para este laboratorio. El objetivo es reducir o evitar la pérdida de paquetes por saturación de buffers sin asumir que el generador puede cambiar mágicamente su tasa de producción.

En esta parte la idea central es que `TransportTx` no envíe todo lo que recibe del generador inmediatamente. En cambio, debe mantener un buffer de transmisión y regular cuántos paquetes puede tener "en vuelo" usando una ventana efectiva de envío.

### 1. Cambios en la red

Agreguen un canal de retorno desde `nodeRx` hacia `nodeTx`. Ese canal debe permitir enviar información de control desde el receptor hacia el transmisor.

Para esta parte, conviene reemplazar las colas internas de los nodos por módulos especializados:

- `TransportTx`: buffer del transmisor y lógica de regulación.
- `TransportRx`: buffer del receptor y lógica de feedback.

Un esquema posible es:

datos:

```
nodeTx.TransportTx -> queue -> nodeRx.TransportRx -> Sink
```

feedback:

```
nodeRx.TransportRx -> feedbackQueue -> nodeTx.TransportTx
```

Para implementar enlaces bidireccionales en OMNeT++, pueden usar gates `inout` y acceder a sus direcciones con `$i` y `$o`.

Ejemplo mínimo de los módulos simples de transporte:

```
simple TransportTx
{
    parameters:
        int bufferSize;
    gates:
        input toApp;
        inout toOut;
}

simple TransportRx
{
    parameters:
        int bufferSize;
```

```

    gates:
        output toApp;
        inout toOut;
}

```

Los nodos compuestos también deben exponer gates inout:

```

module NodeTx
{
    gates:
        inout out;
    submodules:
        gen: Generator;
        transport: TransportTx;
    connections:
        gen.out --> transport.toApp;
        transport.toOut <--> out;
}

module NodeRx
{
    gates:
        inout in;
    submodules:
        transport: TransportRx;
        sink: Sink;
    connections:
        in <--> transport.toOut;
        transport.toApp --> sink.in;
}

```

Ejemplo de conexiones con gates inout:

```

submodules:
    nodeTx: NodeTx;
    queue: Queue;
    feedbackQueue: Queue;
    nodeRx: NodeRx;

connections:
    nodeTx.out$o --> { datarate = 1Mbps; delay = 100us; } --> queue.in;
    queue.out --> { datarate = parent.queueToRxDatarate; delay = 100us; } --> nodeRx.in$i;

    nodeRx.in$o --> { datarate = 1Mbps; delay = 100us; } --> feedbackQueue.in;
    feedbackQueue.out --> { datarate = 1Mbps; delay = 100us; } --> nodeTx.out$i;

```

## 2. Ventanas obligatorias

La implementación de la Parte 2 debe usar, como mínimo, estas tres variables conceptuales:

- **rwnd**: ventana de recepción. La calcula TransportRx según el espacio libre en su buffer.
- **cwnd**: ventana de congestión. La mantiene TransportTx según señales de congestión de la

red.

- `effectiveWindow`: ventana efectiva de envío.

La regla principal es:

```
effectiveWindow = min(rwnd, cwnd);
```

TransportTx sólo puede enviar nuevos paquetes si la cantidad de paquetes en vuelo es menor que la ventana efectiva:

```
while (inFlight < min(rwnd, cwnd) && !txBuffer.isEmpty()) {
    sendNextPacket();
}
```

Donde `inFlight` representa paquetes enviados por TransportTx que todavía no fueron reconocidos mediante feedback.

Esta regla implementa dos controles al mismo tiempo:

- Control de flujo: si el receptor tiene poco espacio, anuncia un `rwnd` chico.
- Control de congestión: si la red muestra señales de congestión, el transmisor reduce `cwnd`.

No se pide implementar TCP completo. No hacen falta números de secuencia perfectos, retransmisión completa, fast retransmit ni RTT estimado con precisión. Sí se pide implementar una versión coherente y medible de este mecanismo de ventanas.

### 3. Paquetes de datos y feedback

Pueden distinguir tipos de paquetes usando `setKind` / `getKind`:

```
enum PacketKind {
    DATA = 1,
    ACK = 2,
    WINDOW_UPDATE = 3
};
```

Al crear paquetes de datos:

```
cPacket *pkt = new cPacket("packet");
pkt->setByteLength(par("packetByteSize"));
pkt->setKind(DATA);
```

Al crear un ACK o actualización de ventana:

```
cPacket *ack = new cPacket("ack");
ack->setByteLength(20);
ack->setKind(ACK);
ack->addPar("rwnd") = freeBufferSlots;
ack->addPar("ecn") = congestionMarked;
send(ack, "toOut$o");
```

`rwnd` debe indicar cuántos paquetes puede aceptar el receptor en ese momento. `ecn` puede usarse como una señal explícita de congestión: por ejemplo, una cola intermedia puede marcar paquetes cuando su ocupación supera cierto umbral.

En el transmisor, el feedback debe actualizar la ventana de recepción y la ventana de congestión:

```

if (msg->getKind() == ACK || msg->getKind() == WINDOW_UPDATE) {
    rwnd = (int) msg->par("rwnd").longValue();

    if (msg->getKind() == ACK) {
        inFlight--;
    }

    if (msg->hasPar("ecn") && msg->par("ecn").boolValue()) {
        cwnd = std::max(1.0, cwnd / 2.0);
    } else if (msg->getKind() == ACK) {
        cwnd += 1.0 / cwnd;
    }

    delete msg;
    return;
}

```

El crecimiento de cwnd puede variar. Una opción simple es crecimiento aditivo y reducción multiplicativa:

```

// Si no hay congestión:
cwnd += 1.0 / cwnd;

// Si hay congestión:
cwnd = std::max(1.0, cwnd / 2.0);

```

Si prefieren, pueden definir un archivo `packet.msg` con campos propios, por ejemplo `seq`, `ack`, `rwnd`, `ecn`, `timestamp`, o cualquier otro dato útil para el protocolo.

#### 4. Pistas prácticas de OMNeT++

Hay algunos detalles de implementación que suelen aparecer recién cuando la simulación empieza a correr. Ténganlos presentes:

**No envíen por un canal ocupado.** Si un módulo intenta hacer `send()` por una gate cuyo canal de transmisión todavía está ocupado, OMNeT++ va a abortar la simulación con un error. Antes de enviar, consulten el canal de salida:

```

cChannel *channel = gate("out")->findTransmissionChannel();

if (channel && channel->isBusy()) {
    scheduleAt(channel->getTransmissionFinishTime(), sendEvent);
    return;
}

send(pkt, "out");

```

Esto aplica tanto a paquetes de datos como a feedback. En particular, el receptor puede generar ACKs más rápido de lo que el canal de retorno puede transmitirlos.

**Usen colas internas para serializar envíos.** Si un módulo puede tener varios paquetes listos para salir por la misma gate, guárdenlos en una `cQueue` y usen un self-message para enviar el próximo cuando el canal esté disponible:

```
feedbackQueue.insert(ack);

if (!sendFeedbackEvent->isScheduled()) {
    simtime_t when = simTime();
    cChannel *channel = gate("toOut$o")->findTransmissionChannel();
    if (channel && channel->isBusy()) {
        when = channel->getTransmissionFinishTime();
    }
    scheduleAt(when, sendFeedbackEvent);
}
```

**No asuman que los parámetros de un cPacket usan la misma API que los parámetros del .ini.** Si agregan campos con addPar, OMNeT++ devuelve un cMsgPar. Para leer enteros usen longValue() y para booleanos boolValue():

```
int rwnd = (int) msg->par("rwnd").longValue();
bool ecn = msg->hasPar("ecn") && msg->par("ecn").boolValue();
```

**En canales definidos en NED, referencien parámetros del módulo padre con parent.** Dentro del bloque de un canal, expresiones como linkDelay o queueToRxDataRate pueden no resolver como esperan. Usen:

```
queue.out --> {
    datarate = parent.queueToRxDataRate;
    delay = parent.linkDelay;
} --> nodeRx.in$i;
```

**La ventana limita paquetes en vuelo, no paquetes generados.** El generador puede seguir produciendo paquetes. El lugar donde se regula el flujo es TransportTx: si inFlight >= min(rwnd, cwnd), el paquete debe esperar en el buffer de transmisión.

## 5. Señales de congestión

Cada grupo debe definir cómo detecta congestión. Algunas opciones razonables:

- Marcar paquetes con ecn=true cuando la cola intermedia supera un umbral, por ejemplo 80% del buffer.
- Reducir cwnd cuando hay pérdidas en la cola intermedia.
- Reducir cwnd cuando la demora promedio o el tiempo de vida de los paquetes crece demasiado.
- Usar timeouts simples si un paquete enviado no recibe ACK dentro de cierto tiempo.

El receptor puede reenviar al transmisor la señal de congestión recibida en los paquetes de datos:

```
bool congestionMarked = pkt->hasPar("ecn") && pkt->par("ecn").boolValue();

cPacket *ack = new cPacket("ack");
ack->setKind(ACK);
ack->addPar("rwnd") = bufferSize - buffer.getLength();
ack->addPar("ecn") = congestionMarked;
send(ack, "toOut$o");
```

## 6. Libertades de diseño

El piso obligatorio es usar `rwnd`, `cwnd`, `inFlight` y `min(rwnd, cwnd)`. Sobre ese piso, pueden tomar decisiones propias:

- Cómo inicializan `cwnd`.
- Cómo crece `cwnd` cuando no hay congestión.
- Cómo decrece `cwnd` cuando aparece congestión.
- Si usan ACK por paquete, ACK acumulativo o feedback periódico.
- Si implementan retransmisión o solamente regulan el envío.
- Cómo evitan oscilaciones fuertes.
- Qué métricas internas registran para explicar el comportamiento.

El informe debe explicar claramente estas decisiones. No alcanza con decir "es como TCP": deben indicar qué parte de TCP simplificaron, qué conservaron y qué consecuencias tiene.

## 7. Experimentos de la Parte 2

Usen los mismos parámetros y casos de la Parte 1. Comparen:

- Caso 1 sin control vs. Caso 1 con control por ventanas.
- Caso 2 sin control vs. Caso 2 con control por ventanas.

Generen nuevamente las gráficas necesarias para ver:

- Ocupación de buffers.
- Pérdidas de paquetes.
- Carga ofrecida vs. carga útil.
- Demora, si ayuda a explicar el comportamiento.
- Evolución de `rwnd`, `cwnd`, `effectiveWindow` e `inFlight`.

La gráfica de carga ofrecida vs. carga útil debería permitir comparar al menos cuatro curvas:

- Caso 1 sin control.
- Caso 2 sin control.
- Caso 1 con control por ventanas.
- Caso 2 con control por ventanas.

Si agregan una curva de límite teórico, expliquen cómo la calcularon.

## 8. Preguntas de la Parte 2

Respondan y justifiquen:

1. ¿Cómo implementaron `rwnd`, `cwnd` y `effectiveWindow`?
2. ¿Qué información viaja en el feedback?
3. ¿Qué condición usa el transmisor para enviar o esperar?
4. ¿Cómo detectan congestión?
5. ¿Cómo cambia `cwnd` cuando hay congestión y cuando no la hay?
6. ¿El algoritmo funciona igual para el Caso 1 y el Caso 2? ¿Por qué?
7. ¿Logra reducir o eliminar pérdidas?
8. ¿Qué costo tiene la solución? Por ejemplo: mayor demora, menor carga útil, oscilaciones, subutilización del enlace.
9. ¿Qué limitaciones tiene su diseño y cómo podría mejorarse?

## Competición de control de transporte

Al finalizar la Parte 2 vamos a comparar los algoritmos de todos los grupos en un mismo escenario. La idea de la competición no es reemplazar el informe: es una forma de discutir los trade-offs entre pérdida, demora, utilización y estabilidad.

### Escenario oficial

Cada grupo debe reportar las métricas de la competición usando exactamente este setup:

```
[Config Competition]
description = "Escenario oficial para comparar control de transporte"
sim-time-limit = 200s

Network.nodeTx.gen.packetByteSize = 12500
Network.nodeTx.gen.generationInterval = exponential(0.1)

Network.nodeTx.transport.bufferSize = 2000000
Network.queue.bufferSize = 100
Network.feedbackQueue.bufferSize = 1000
Network.nodeRx.transport.bufferSize = 100

Network.txToQueueDatarate = 1Mbps
Network.queueToRxDatarate = 0.5Mbps
Network.nodeRx.rxToSinkDatarate = 0.5Mbps
Network.feedbackDatarate = 1Mbps
Network.linkDelay = 100us
```

Este escenario combina presión sobre la red y sobre el receptor. Si sus nombres de módulos son distintos, adapten los paths, pero no cambien los valores.

### Métricas oficiales

Cada grupo debe reportar un único valor para cada métrica, calculado sobre toda la simulación de 200s:

| Métrica   | Unidad     | Cómo calcularla                                                 |
|-----------|------------|-----------------------------------------------------------------|
| goodput   | paquetes/s | paquetes recibidos por Sink / 200s                              |
| lossRate  | porcentaje | $100 * \text{paquetes descartados} / \text{paquetes generados}$ |
| avgDelay  | segundos   | demora promedio extremo a extremo de paquetes recibidos         |
| stability | paquetes   | desviación estándar de effectiveWindow                          |

Para que el scoreboard sea fácil de leer, el informe debe incluir una tabla con este formato:

```
Grupo | goodput (pkt/s) | lossRate (%) | avgDelay (s) | stability (pkt)
```

No hay una única forma correcta de ganar: un algoritmo puede lograr mucho **goodput** con mucha demora, o muy baja pérdida con baja utilización. La discusión de esos compromisos forma parte de la evaluación.

## Reproducibilidad

Junto con la tabla, indiquen el comando usado para correr la simulación y asegúrense de que el repositorio permita reproducir esos cuatro valores sin editar el código a mano.

## Informe

Entreguen un informe en texto plano o Markdown. La estructura sugerida es:

1. Título e integrantes.
2. Resumen breve.
3. Introducción: problema a estudiar.
4. Métodos: modelo, parámetros, modificaciones y algoritmo propuesto.
5. Resultados: gráficas y observaciones.
6. Discusión: interpretación, limitaciones y posibles mejoras.
7. Conclusiones.
8. Anexo sobre uso de herramientas de Inteligencia Artificial.

El anexo de IA debe indicar:

- Qué herramientas usaron, si usaron alguna.
- Para qué las usaron.
- Qué prompts o interacciones fueron relevantes.
- Cómo validaron las respuestas obtenidas.

## Entrega Final

El laboratorio tiene una entrega intermedia al finalizar la Parte 1 y una entrega final al completar la Parte 2.

La entrega intermedia debe incluir:

- Código fuente correspondiente a la Parte 1.
- `network.ned` y `omnetpp.ini` con los dos casos de estudio reproducibles.
- Métricas y gráficas usadas para analizar ocupación de buffers, demoras y pérdidas.
- Respuestas a las preguntas de la Parte 1.

La entrega final debe incluir:

- Código fuente del modelo OMNeT++.
- `network.ned` y `omnetpp.ini` con configuraciones reproducibles.
- Informe en Markdown o texto plano.
- Gráficas usadas en el informe, si no están embebidas.
- Tabla de la competición con `goodput`, `lossRate`, `avgDelay` y `stability` para el escenario oficial.
- Cualquier script auxiliar usado para procesar resultados, si corresponde.

La entrega se realiza por el repositorio provisto por la cátedra, con la fecha límite indicada en el cronograma del aula virtual.

## Criterios de evaluación

Se evaluará:

- Claridad y legibilidad del código.
- Correcta modelización de paquetes, tasas y buffers.
- Registro adecuado de métricas.



- Experimentos reproducibles.
- Calidad del análisis, no solamente cantidad de gráficas.
- Coherencia del algoritmo de ventanas y feedback.
- Claridad en el reporte de métricas de la competición.
- Discusión honesta de limitaciones y resultados inesperados.

### Consejos y errores frecuentes

- No usen `cMessage` para paquetes de datos si necesitan tamaño en bytes; usen `cPacket`.
- Todo paquete descartado debe borrarse con `delete` para evitar fugas de memoria.
- No alcanza con mirar la demora promedio: miren en qué buffer crece la ocupación.
- Mantengan separados los resultados de Caso 1 y Caso 2.
- Usen nombres de vectores claros, por ejemplo `bufferSize`, `packetDrops`, `packetGen`.
- Verifiquen que `generationInterval` represente la media de una distribución exponencial.
- Eviten cambiar manualmente el `.ned` entre corridas si pueden resolverlo con configuraciones en `omnetpp.ini`.
- Si su algoritmo no elimina todas las pérdidas, no significa necesariamente que el laboratorio esté mal: expliquen cuándo falla y por qué.

### Comandos útiles

Compilar:

```
make
```

Correr una configuración por consola:

```
./lab3-kickstarter -u Cmdenv -c Caso1  
./lab3-kickstarter -u Cmdenv -c Caso2
```

Limpiar archivos de compilación:

```
make clean
```